

```

import serial
import time
import os
PORT = "COM20"
BAUD = 260000
PACKET_SIZE = 128

ser = serial.Serial(PORT, BAUD)
time.sleep(2)
ser.reset_input_buffer()
ser.reset_output_buffer()

path = r"C:\Users\Abhay Verma\Desktop\fpic_mic_1.txt"
data = open(path, "rb").read()
filesize = len(data)
print("File size:", filesize)

def checksum(packet):
    return sum(packet) % 256

ser.write(b"abc\n")
time.sleep(0.01)
ser.write(b"abc\n")
ser.write(b"<START>\n")
ser.write(str(filesize).encode() + b"\n")
ser.write(str(PACKET_SIZE).encode() + b"\n")

packet_no = 0

for i in range(0, filesize, PACKET_SIZE):
    chunk = data[i:i+PACKET_SIZE]
    header = bytes([0xAA, 0x55])
    length = len(chunk)
    packet = header + bytes([packet_no]) + bytes([length]) + chunk
    crc = checksum(packet)
    packet += bytes([crc])
    ser.write(packet)
    packet_no = (packet_no + 1) % 256
    #time.sleep(0.01)
    ser.flush()

ser.write(b"<END>\n")
print("Transmission complete")
ser.close()

```

Fig. 4: Snippet of the Python program used in data transmission

```

import serial
import time
PORT = "COM31"
BAUD = 260000

ser = serial.Serial(PORT, BAUD)
time.sleep(2)
print("Waiting for start")

line = ser.readline()

while b"<START>" not in line:
    line = ser.readline()

print("Start detected")

filesize = int(ser.readline())
packet_size = int(ser.readline())
print("File size:", filesize)
print("Packet size:", packet_size)

received = 0
file = open(r"C:\Users\EFY\Desktop\received.zip", "wb")

def checksum(packet):
    return sum(packet) % 256

while received < filesize:
    header = ser.read(2)
    if header != b'\xAA\x55':
        continue
    packet_no = ser.read(1)
    length = ser.read(1)[0]
    data = ser.read(length)
    crc_rx = ser.read(1)[0]
    packet = header + packet_no + bytes([length]) + data
    crc_calc = checksum(packet)
    if crc_calc == crc_rx:
        file.write(data)
        received += length
        #print("Received:", received)
        if received % 4096 == 0:
            print("Received:", received)
    else:
        print("CRC error")

file.close()
print("File received successfully")

```

Fig. 5: Snippet of the Python receiver program used for Li-Fi

section of the LM393 is not used in this system.

The IC1B section of the LM393 converts the weak analogue signal from the photodiode into clean digital pulses suitable for UART communication. Since the comparator has an open-collector output, the pull-up resistor R2 is used to ensure a proper logic-high level. Capacitor C1 provides supply filtering and improves noise immunity for stable high-speed operation.

During operation, variations in the received laser light cause the comparator IC1B to switch its output accordingly, thereby reconstructing the transmitted binary data. The recovered digital signal is then fed to the RXD pin of the CH9102 USB-to-UART module for transfer to the computer through the USB interface.

The CH9102 USB-to-UART module does not require any specialised software. Its serial interface can be accessed through Python using terminal applications such as Command Prompt.

Software

The Python script implements the transmitter side of the Li-Fi communication system. It reads a file from the computer, divides the data into packets, and transmits it through a serial (UART) interface connected to the optical transmitter setup. The program establishes communication over a specified COM port at a high baud rate of 260000 and opens the file in binary mode, allowing transmission of any type of digital file.

Before transmitting the actual data, the program performs a simple handshake using dummy synchronisation signals followed by a

marker. It then sends metadata such as the file size and packet size to the receiver. The file is divided into fixed-size packets of 128 bytes. Each packet contains a header (0xAA 0x55), packet number, data length, payload data, and a checksum for basic error detection.

The program uses the pyserial library to communicate with the CH9102 USB-to-UART module connected to the laser transmitter circuit. During transmission, packets are sent sequentially through the serial interface, and the packet number is incremented modulo 256 to maintain the correct order. A simple checksum, generated by summing bytes modulo 256 is appended to each packet so the receiver can verify data integrity. The flush() function is used to ensure the timely delivery of data, which is important for reliable optical communication.

After opening the serial port and completing synchronisation, the transmitter sends the file size and packet information, followed by the packetised data stream. The serial data modulates the laser diode in the Li-Fi transmitter circuit using on-off keying (OOK), where the on state of the laser represents a logic high (1) and the off state represents a logic low (0).

Once all packets have been transmitted successfully, the program sends a marker to indicate the completion of the data transfer and then safely closes the serial connection. Fig. 4 shows a snippet of the Python program used to transmit Li-Fi serial data via the laser transmitter module.

The COM port must be configured correctly before running the Python script. After connecting the CH9102 module to the PC, open Device Manager and check the assigned port number under the Ports (COM and LPT) section. For example, COM20.

Also, ensure that the correct file paths are selected for both the test file and the received file. In this example, both files are placed inside the Desktop folder, and the Python script is executed from the same